

Лабораторная работа №3. Синтаксический анализ текста в формате JSON.

1. Цель и содержание

Цель лабораторной работы: Научиться проводить синтаксический анализ текста в формате json.

2. Теоретическая часть

Каждый язык программирования описывается с помощью набора правил, определяющих структуру правильной программы. Удобным формализмом для описания синтаксических конструкций языка программирования являются контекстно-свободные грамматики.

Граматики одинаково помогают решать задачи как программистов, использующих язык, так и создателей компиляторов для данного языка:

- Грамматика предоставляет точную и достаточно легкую для понимания синтаксическую спецификацию языка программирования.
- Для некоторых классов грамматик мы можем автоматически сконструировать эффективный анализатор, который определяет, является ли исходная программа синтаксически правильной.
- Аккуратно созданная грамматика может придать языку программирования такую структуру, которая будет полезна и при трансляции исходной программы в правильный объектный код, и при определении ошибок.
- Компиляторы, разработанные на базе грамматик, могут быть достаточно легко расширены (это особенно полезно для добавления новых конструкций, появившихся как результат развития языка)

С помощью контекстно-свободных грамматик определяется только так называемая контекстно-свободная составляющая языка программирования, то есть только то, каким образом записывается та или иная конструкция языка. Другая важная часть определения синтаксической правильности программы – правильность использования типов в программе – не может быть определена с помощью контекстно-свободных грамматик. Поэтому если

программа выводима в грамматике, это еще не означает, что она полностью синтаксически правильна.

Синтаксический анализ – это процесс, который определяет, принадлежит ли некоторая последовательность лексем языку, порождаемому грамматикой. В принципе, по любой грамматике можно построить синтаксический анализатор, но грамматики, используемые на практике, имеют специальную форму. Например, известно, что для любой контекстно-свободной грамматики может быть построен анализатор, сложность которого не превышает $O(n^3)$ для входной строки длины n , но в большинстве случаев по заданному языку программирования мы можем построить такую грамматику, которая позволит сконструировать и более быстрый анализатор. Анализаторы реально используемых языков обычно имеют линейную сложность; это достигается, например, за счет просмотра исходной программы слева направо с заглядыванием вперед на один терминальный символ (лексический класс).

Вход синтаксического анализатора – последовательность лексических и таблицы, например, таблица внешних представлений, которые являются выходом лексического анализатора.

Выход синтаксического анализатора – дерево разбора и таблицы, например, таблица идентификаторов и таблица типов, которые являются входом для следующего просмотра компилятора (например, это может быть просмотр, осуществляющий контроль типов).

Для выполнения синтаксического анализа построим грамматику входного языка:

```
<программа> ::=PROC MAIN;<текст> END.  
  
<текст> ::= (<строка> | <Процедура>){<строка>| <процедура>}  
  
<строка> ::= <описание>|<отд. оператор> <описание> ::= DCL  
<идентификатор>{,<идентификатор>} DEC FIXED;  
  
<отд. оператор> :=< Идентификатор>: <оператор>|<оператор>  
  
<оператор> ::= <идентификатор>:=<выражение> | GOTO <идентификатор> |  
<условный оператор> <процедура> ::= PROC <идентификатор>;<текст>END;
```

```

<условный оператор> ::= IF <условие> THEN <оператор> <условие .> ::= =
<выражение><неравенство><выражение>

<выражение> ::= <терм>{+<терм>}

<терм> ::= <множитель>{*<множитель>}

<множитель> ::= <аргумент> | (<«выражение»>)

<аргумент> ::= <идентификатор> | <константа>

<идентификатор> ::= <буква>{<буква>|<цифра>}

<константа> ::= <число> | .<число> | <число>.<число>

<буква> ::= A | B | ... | Y | Z

<цифра> ::= 0 | 1 | ... | 8 | 9

```

2.1 Метод рекурсивного спуска

Одним из наиболее простых и потому одним из наиболее популярных методов нисходящего синтаксического анализа является метод рекурсивного спуска.

Для объяснения принципов, лежащих в основе метода рекурсивного спуска, рассмотрим задачу вычисления значения арифметической формулы. Будем рассматривать формулы, состоящие из целочисленных значений, бинарных операций сложения (+), вычитания (−), умножения (*) и деления (/), а также круглых скобок. Как обычно, приоритеты операций умножения и деления равны и их приоритет больше, чем приоритеты операций сложения и вычитания, причем приоритеты этих операций также равны. Будем называть операции + и − операциями типа сложения, а операции * и / – операциями типа умножения. Круглые скобки используются для изменения стандартного порядка выполнения операций. Наша задача заключается в написании программы, вычисляющей значение формулы.

Изучаемые нами формулы можно представить следующим образом:

$$T_1 + T_2 + \dots + T_n,$$

где T_i – это формула вида $F_{1i} * F_{2i} * \dots * F_{mi}$. В свою очередь, F_{ji} – это либо число, либо произвольная формула, заключенная в круглые скобки.

Представим себе процесс вычисления значения формулы. Вначале вычисляется F_{11} , далее мы выясняем, какая операция следует за F_{11} . Если это

операция типа умножения, то мы, зная ее левый операнд, вычисляем правый операнд и выполняем операцию. Тем самым, мы получим левый операнд для возможных следующих операций типа умножения. Когда мы закончим вычисление формулы $F_1 * F_2 * \dots * F_n$, то, возможно, увидим далее операцию типа сложения, и процесс вычисления такой формулы будет аналогичен только что описанному процессу.

Рассмотрим синтаксический анализ методом рекурсивного спуска применительно к анализу грамматики исходного языка программирования.

В соответствии с методом рекурсивного спуска для каждого нетерминала создается рекурсивная процедура, и каждая такая процедура осуществляет разбор фраз для данного нетерминала. Процедура ищет фразу, сравнивая программу в указанном месте с правыми частями правил для данного нетерминала, вызывая, если нужно, другие процедуры для промежуточных целей.

Для того, чтобы не было возвратов, в качестве контекста используется единственный символ, следующий за разобранной частью фразы. В алгоритме каждому нетерминалу соответствует одна рекурсивная процедура, каждая процедура осуществляет разбор фраз, выводимых из соответствующего нетерминала. Процедура для нетерминала U ищет фразу следующим образом: сравнивает разбираемый символ с правыми частями правил для данного нетерминала и вызывает другие процедуры по мере необходимости. Для того, чтобы работал алгоритм, необходимо определить некоторые понятия.

Введем переменную `NXTSYMB` – глобальную переменную, содержащую символ, следующий за разбираемым. `NXTSYMB` содержит символ (лексему) исходной программы, который будет обрабатываться следующим и при поиске новой цели первый символ, который должен быть исследован, всегда находится в переменной `NXTSYMB`.

При выходе из процедуры с сообщением об успехе символ, следующий за закрытой подцепочкой, помещается в `NXTSYMB`.

Имеется специальная процедура SCAM, которая готовит очередной символ исходной программы и помещает его в NXTSYMB.

Предусмотрена процедура ERROR для обработки ошибочных ситуаций.

Ниже в таблице 1 приведены нетерминалы грамматики и имена соответствующих им рекурсивных процедур.

Процедуры IDENT и CONST в отличие от всех остальных возвращают значение ИСТИНА или ЛОЖЬ в зависимости от того, является ли лексема, содержащаяся в переменной NXTSYMB идентификатором или константой, соответственно.

При инициализации синтаксического анализатора идет обращение к процедуре SCAN, которая помещает первый символ исходной программы в глобальную переменную NXTSYMB и вызывает процедуру PROGRAM.

Таблица 1 – Соответствие нетерминалов грамматики и имен рекурсивных процедур

Нетерминальные символы грамматики	Имена рекурсивных процедур
<программа>	PROGRAM
<текст>	TEXT
<строка>	LINE
<описание>	DECLARE
<отд. оператор>	SEOPER
<оператор>	OPERATOR
<процедура>	PROCED
<условный оператор>	IFOPER
<условие>	CONDITION
<выражение>	EXPRESSION
<терм>	TERM
<множитель>	FACTOR
<аргумент>	ARGUMENT

<идентификатор>	IDENT
<константа>	CONST

```

procedure PROGRAM;
begin
  SCAN;
  If NXTSYMB<>'PROC' then ERROR:
  SCAN;
  If NXTSYMB<>'MAIN' then ERROR;
  If NXTSYMB<>';' then ERROR;
  SCAN:
  TEXT;
  SCAN;
  If NXTSYMB<>'END' then ERROR:
  SCAM:
  If NXTSYMB<>'.' then ERROR;
  enf.

procedure TEXT;
while NXTSYMB<>'END' do
  If NXTSYMB<>'PROF' then PROCED else LINE;
procedure PROCED;
begin
  SCAN;
  If NOT IDENT (NXT3YMB) then ERROR;
  SCAN;
  If NXTSYMB<>';' then ERROR;
  SCAN;
  while NXTSYMB<>'END' do begin LINE:
  SCAN:
  end;
  SCAN;
  If NXTSYMB<>';' then ERROR;
  end;
procedure LINE;
  If NXTSYMB='DCL' then DECLARE
  else SEP_OPER:
  procedure DECLARE;
  begin SCAN;
  If NOT IDENT (NXTSYMB) then ERROR;
  SCAN;
  while NXTSYMB=',' do begin
  SCAN;
  If NOT IDENT (NXTSYMB) then ERROR;
  SCAN:
  end;
  If NXTSYMB<>'DEC' then ERROR;
  SCAN;
  If NXTSYMB<>'FIXED' then ERROR;
  SCAN;

  If NXTSYMB<>';' then ERROR;
  SCAN;
  end:
procedure SEP_OPER;
  If NXTSYMB='If' OR NXTSYMB='GOTO' then OPERATOR else begin
  If NOT IDENT (NXTSYMB) then ERROR;
  SCAN:
  If NXTSYMB=':' then begin SCAN;
    OPERATOR;
  end;

```

```

else If NXTSYMB=':=' then begin SCAN;
EXPRESSION;
end;
else
ERROR;
procedure OPERATOR;
If NXTSYHB='If' then If_OPER
else If NXTSYMB='GOTO' then begin SCAN;
If NOT IDENT (NXTSYMB) then ERROR;
end;
else begin
IDENTIFIER;
If NXTSYMB<>':=' then ERROR else begin SCAN;
EXPRESSION;
end;
end;
procedure If_OPER;
begin SCAN;

CONDITION;
If NXTSYMB<>'THEN' then ERROR else begin SCAB:
OPERATOR;
end;
end;
procedure CONDITION:
begin:
EXPRESSION;
If NXTSYMB<>'>' then
If NXTSYMB<>'<' then ERROR;
SCAN;
EXPRESSION;
end;
procedure EXPRESSION;
begin TERM;
SCAN;
while NXTSYMB='+' do begin SCAN;
FACTOR:
end;
end;
procedure TERM:
begin
FACTOR;
while NXTSYNB='*' do begin SCAN:
FACTOR;
end;
end;
procedure FACTOR:
If NXTSYMB='(' then begin
SCAN
EXPRESSION.
If NXTSYMB<>')' then ERROR;
else
SCAN:
end; else
begin
ARGUMENT;
SCAN; end;
procedure ARGUMENT; If NOT CONST(NXTSYMB) then
If NOT IDENT(NXTSYMB) then ERROR;

```

5. Содержание отчета и его форма

1. Номер и название лабораторной работы.

2. Цели лабораторной работы.
3. Ответы на контрольные вопросы.
4. Экранные формы, показывающие порядок выполнения лабораторной работы, и результаты, полученные в ходе её выполнения:

Отчет о выполнении лабораторной работы в письменном виде сдается преподавателю.

6. Задание

1. Разработайте программу, которая выполняет синтаксический анализ текста в формате json.
2. Программа должна в консоль выводить описание ошибки в понятном виде, номер строки в которой содержится ошибка и часть json файла (2 строки до ошибки, строку ошибки и 2 строки после ошибки).
3. В качестве исходного файла используйте файлы lr3*.json, где звездочка – номера от 1 до 5.
4. Для разработки программы запрещается использовать какие-либо дополнительные библиотеки или программы (кроме стандартных в используемом вами ЯП).

7. Защита работы

Перед выполнением лабораторной работы каждый студент получает индивидуальное задание.

Защита лабораторной работы происходит только после его выполнения (индивидуального задания). При защите лабораторной работы студент:

- 1) отвечает на контрольные вопросы;
- 2) поясняет ход выполнения индивидуального задания;
- 3) поясняет результаты, полученные в результате выполнения индивидуального задания.

Ход защиты лабораторной работы контролируется преподавателем.